

Informe de Performance

1. Introducción

El presente informe describe el rendimiento de las cuatro estructuras de datos desarrolladas en el Taller 3: pilas, colas, listas enlazadas y árboles AVL. En cada ejercicio se aplicó una estructura diferente para resolver un problema usando datos reales.

2. Pilas

En el presente ejercicio se utilizó una pila para determinar el stock span de los precios de la bolsa orientada a un análisis de datos bursátiles. El stock span indica el número de días consecutivos hacia atrás en los cuales el precio era menor o igual al precio.

La pila almacena índices de los días. Si el precio actual es mayor o igual al precio que está en el tope de la pila, se desapila hasta encontrar un precio mayor.

La complejidad del algoritmo es $O(n)$ debido a que cada elemento se apila y desapila como máximo una vez.

Resultado obtenido:

Registros: 1970

Span máximo: 1970 días

Fecha: 2017-11-10

Conclusión:

La pila permitió resolver el problema de forma eficiente, evitando comparar cada día con todos los días anteriores.

3. Colas

En este ejercicio se implementó un limitador de peticiones usando colas. Cada IP tiene una cola donde se guardan los tiempos de sus peticiones.

La regla usada fue permitir como máximo M peticiones dentro de una ventana de T segundos. Si una IP supera ese límite, la petición se rechaza.

La complejidad es $O(n)$, porque se recorre el archivo una sola vez. Las operaciones de la cola son $O(1)$.

Resultado obtenido:

Peticiones: 19

$M = 2$

$T = 60$ segundos

Total de rechazos: 11

IP con más rechazos: 2.177.12.140

Conclusión:

La cola permite controlar las peticiones de cada IP de manera eficiente, eliminando los registros antiguos de la ventana de tiempo.

4. Listas Enlazadas

En este ejercicio fue implementada una caché LRU con una lista doblemente enlazada más un mapa.

La lista mantiene el orden de uso de los elementos. El elemento más reciente se coloca al inicio y el menos usado queda al final. Así, al llenarse la caché, se elimina el elemento menos usado.

El mapa permite acceder rápidamente a los elementos y la lista permite moverlos o eliminarlos en tiempo constante.

La complejidad de Get y Put es $O(1)$.

Resultado obtenido:

Accesos cargados: 100836

Tamaño de caché	Hit ratio
50	0.0285
100	0.0692
500	0.3226
1000	0.5351

Conclusión:

Mientras mayor es la capacidad de la caché, mayor es el hit ratio. Esto ocurre porque la caché puede guardar más elementos recientes.

5. Árboles AVL

En este ejercicio se implementó un árbol AVL para indexar películas usando el año como clave. Cada nodo representa un año y guarda todas las películas de ese año.

El AVL se mantiene balanceado usando rotaciones:

- LL
- RR
- LR
- RL

Luego, se realizan consultas por rango. Por ejemplo, buscar todas las películas entre 1995 y 2000.

La inserción en el AVL tiene complejidad $O(\log n)$ y la consulta por rango tiene complejidad $O(\log n + k)$, donde k es la cantidad de resultados encontrados.

Resultado obtenido:

Registros indexados: 86968

Nodos AVL: 142

Altura del árbol: 8

Rotaciones totales: 93

Películas entre 1995 y 2000: 5754

Conclusión:

El árbol AVL evita que el árbol se desbalancee. Gracias a esto, las búsquedas y consultas por rango se realizan de manera eficiente.

6. Comparación General

Estructura	Complejidad principal	Resultado
Pila	$O(n)$	Span máximo de 1970 días
Colas	$O(n)$	11 rechazos
Listas	$O(1)$	Hit ratio máximo de 0.5351
Árbol AVL	$O(\log n)$	Altura 8 con 142 nodos